

Quick Sort

1 Introduction

Quick Sort is a **Divide and Conquer** sorting algorithm.

The main idea is:

1. Select an element as the **pivot**.
2. Partition the array around the pivot.
3. Elements smaller than the pivot are placed on the left.
4. Elements greater than the pivot are placed on the right.
5. Recursively apply Quick Sort to the left and right parts.

In this note, we will always take the **first element as the pivot**.

$$\boxed{\text{Pivot} = A[\text{low}]}$$

The array is repeatedly divided until each part contains only one element.

2 Important Variables

We use three important variables:

Variable	Meaning
<i>low</i>	Starting index of the current part
<i>up</i>	Ending index of the current part
<i>mid</i>	Final position of the pivot after partitioning
<i>pivot</i>	First element of the current part

Initially,

$$low = 0$$

and

$$up = n - 1$$

The pivot is:

$$pivot = A[low]$$

After partitioning, the pivot reaches its correct position.
That position is called:

$$\boxed{mid}$$

Therefore, the two recursive parts are:

$$[low, mid - 1]$$

and

$$[mid + 1, up]$$

3 Basic Idea of Partitioning

Suppose the array is:

$$36, 51, 16, 26, 81, 21, 91, 46$$

We take the first element as pivot.

$$pivot = 36$$

Initially:

$$low = 0, \quad up = 7$$

We also use:

$$i = low + 1$$

and

$$j = up$$

The purpose of the two pointers is:

- i moves from left to right.
- j moves from right to left.
- i looks for an element greater than the pivot.
- j looks for an element smaller than the pivot.
- These two elements are exchanged.

4 Using Infinity as a Sentinel

To simplify the rightward search, we place an infinity symbol after the last element.

Original array:

36, 51, 16, 26, 81, 21, 91, 46

After adding infinity:

36, 51, 16, 26, 81, 21, 91, 46, ∞

The infinity element acts as a **sentinel**.

Since every normal number is smaller than ∞ ,

$$A[up + 1] = \infty$$

guarantees that the search from the left will eventually stop.

Important: ∞ is a conceptual sentinel used in the algorithm/pseudocode. It does not mean that an actual C++ integer must contain mathematical infinity.

5 Partition Algorithm

The partition process can be written as follows.

```
Partition(A, low, up)

    pivot = A[low]

    i = low + 1
    j = up

    A[up + 1] = infinity

    while (i <= j)

        while (A[i] <= pivot)
            i++

        while (A[j] > pivot)
            j--

        if (i < j)
            swap(A[i], A[j])

    swap(A[low], A[j])

    mid = j

    return mid
```

After partitioning:

$$A[low \dots mid - 1] \leq A[mid]$$

and

$$A[mid + 1 \dots up] > A[mid]$$

The pivot is now in its correct position.

6 Quick Sort Algorithm

The complete Quick Sort algorithm is:

```
QuickSort(A, low, up)

    if (low < up)

        mid = Partition(A, low, up)

        QuickSort(A, low, mid - 1)

        QuickSort(A, mid + 1, up)
```

The stopping condition is:

$$\boxed{low \geq up}$$

because a part containing zero or one element is already sorted.

7 Example

Consider:

$$A = [36, 51, 16, 26, 81, 21, 91, 46]$$

Number of elements:

$$n = 8$$

Therefore,

$$low = 0$$

$$up = n - 1 = 7$$

The first element is selected as the pivot:

$$pivot = A[low] = 36$$

Add the sentinel:

$$[36, 51, 16, 26, 81, 21, 91, 46, \infty]$$

7.1 Step 1: Initial Values

$$low = 0, \quad up = 7$$

$$pivot = 36$$

$$i = low + 1 = 1$$

$$j = up = 7$$

Array:

36 51 16 26 81 21 91 46

The first element, 36, is the pivot.

7.2 Step 2: Move i

We move i from left to right while:

$$A[i] \leq pivot$$

At $i = 1$:

$$A[1] = 51$$

Since:

$$51 > 36$$

we stop.

Therefore,

$$i = 1$$

7.3 Step 3: Move j

We move j from right to left while:

$$A[j] > pivot$$

At $j = 7$:

$$A[7] = 46$$

Since:

$$46 > 36$$

move j .

At $j = 6$:

$$A[6] = 91$$

Since:

$$91 > 36$$

move j .

At $j = 5$:

$$A[5] = 21$$

Since:

$$21 < 36$$

we stop.

Therefore,

$$j = 5$$

7.4 Step 4: Exchange

We have:

$$i = 1, \quad j = 5$$

Since:

$$i < j$$

exchange:

$$A[1] \leftrightarrow A[5]$$

Therefore:

$$[36, 21, 16, 26, 81, 51, 91, 46]$$

Now continue the scanning.

7.5 Step 5: Continue Scanning

The left pointer i continues from the current position.

The values:

$$21, 16, 26$$

are less than the pivot 36.

Therefore, i moves forward until:

$$A[i] > 36$$

At:

$$i = 4$$

we have:

$$A[4] = 81$$

Since:

$$81 > 36$$

we stop.

Now move j from the right.

At:

$$j = 4$$

we have:

$$A[4] = 81 > 36$$

so j moves left.

At:

$$j = 3$$

we have:

$$A[3] = 26 < 36$$

Therefore,

$$j = 3$$

Now:

$$i = 4, \quad j = 3$$

Since:

$$i > j$$

we stop the partitioning process.

7.6 Step 6: Place the Pivot

The pivot is currently at index $low = 0$.

Exchange:

$$A[low] \leftrightarrow A[j]$$

Therefore:

$$A[0] \leftrightarrow A[3]$$

We get:

$[26, 21, 16, 36, 81, 51, 91, 46]$

The pivot 36 is now at:

$mid = 3$

8 Result of First Partition

Before partitioning:

$[36, 51, 16, 26, 81, 21, 91, 46]$

After partitioning:

$[26, 21, 16, 36, 81, 51, 91, 46]$

The pivot is in its correct position.

Therefore:

$low = 0, \quad mid = 3, \quad up = 7$

The array is divided into:

$[low, mid - 1]$

and

$[mid + 1, up]$

Therefore:

$[0, 2]$

and

$[4, 7]$

So the two parts are:

$[26, 21, 16]$

and

$[81, 51, 91, 46]$

9 Recursive Division

The first partition gives:

$[26, 21, 16] \quad 36 \quad [81, 51, 91, 46]$

Now Quick Sort is applied recursively to both parts.

9.1 Left Part

For:

[26, 21, 16]

we have:

$low = 0, \quad up = 2$

Pivot:

$pivot = 26$

After partitioning:

[16, 21, 26]

Therefore:

$mid = 2$

The left part becomes:

[16, 21]

and the right part is empty.

Now sort:

[16, 21]

Pivot:

16

Since there is only one element on either side after partitioning, this part is sorted.

Thus:

[16, 21, 26]

9.2 Right Part

Consider:

[81, 51, 91, 46]

Here:

$low = 4, \quad up = 7$

Pivot:

$pivot = 81$

After partitioning:

[46, 51, 81, 91]

Therefore:

$mid = 6$

The remaining parts are:

[46, 51]

and

[91]

The first part is sorted using another partition.

Therefore:

[46, 51, 81, 91]

10 Final Result

Combining both sides:

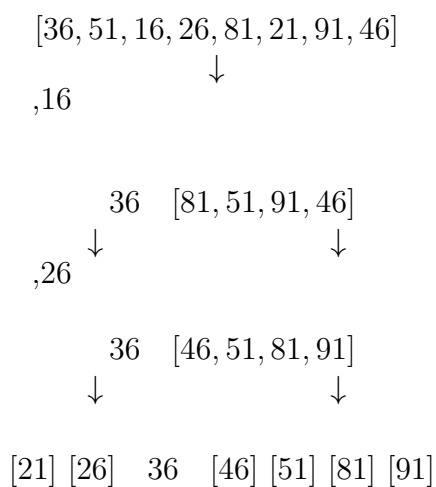
[16, 21, 26] 36 [46, 51, 81, 91]

Hence the final sorted array is:

[16, 21, 26, 36, 46, 51, 81, 91]

11 Quick Sort Division Tree

The division can be represented as:



Each partition places one pivot in its final position.

12 Role of *low*, *mid* and *up*

At every recursive call, we maintain:

low

as the starting position and

up

as the ending position.

After partitioning:

mid

contains the final position of the pivot.

Therefore, the recursive calls are:

$QuickSort(A, low, mid - 1)$

and

$QuickSort(A, mid + 1, up)$

The pivot itself is not included in either recursive call because it is already in its correct position.

13 Time Complexity of Quick Sort

Quick Sort uses the Divide and Conquer technique.

At every partition, the algorithm spends approximately:

$O(n)$

time to divide the current array around the pivot.

The total time depends on how the pivot divides the array.

There are three important cases:

1. Best Case
2. Average Case
3. Worst Case

14 Best Case/ Avg Case

The best case occurs when the pivot divides the array into two approximately equal parts.

For example:

$\frac{n}{2}$ and $\frac{n}{2}$

Therefore, the recurrence relation is:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

where cn represents the time required for partitioning.

14.1 Solving Best Case Recurrence

Start with:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Substitute:

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \frac{n}{2}$$

Therefore:

$$T(n) = 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n$$

$$T(n) = 4T\left(\frac{n}{4}\right) + n + n$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n$$

Again:

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \frac{n}{4}$$

Therefore:

$$T(n) = 8T\left(\frac{n}{8}\right) + 3n$$

After k substitutions:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

The recursion stops when:

$$\frac{n}{2^k} = 1$$

Therefore:

$$n = 2^k$$

Taking logarithm:

$$k = \log_2 n$$

Hence:

$$T(n) = nT(1) + n \log_2 n$$

Therefore:

$$T(n) = O(n \log n)$$

Thus, the best-case time complexity is:

$$O(n \log n)$$

15 Worst Case

The worst case occurs when the selected pivot is always the smallest or largest element.

Since we are taking the **first element as pivot**, the worst case can occur when the array is already sorted or reverse sorted.

For example:

$$[10, 20, 30, 40, 50]$$

If the first element is always selected as pivot:

$$10$$

is the smallest element.

Therefore, the division becomes:

$$0 + (n - 1)$$

instead of:

$$\frac{n}{2} + \frac{n}{2}$$

The recurrence is:

$$T(n) = T(n - 1) + n$$

15.1 Solving Worst Case Recurrence

Start with:

$$T(n) = T(n - 1) + n$$

Substitute:

$$T(n - 1) = T(n - 2) + (n - 1)$$

Therefore:

$$T(n) = T(n - 2) + (n - 1) + n$$

Again:

$$T(n - 2) = T(n - 3) + (n - 2)$$

Therefore:

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

Continuing:

$$T(n) = T(1) + (2 + 3 + \dots + n)$$

Using:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

we get:

$$T(n) = T(1) + \frac{n(n+1)}{2}$$

Ignoring constants and lower-order terms:

$$T(n) = O(n^2)$$

Therefore:

Worst Case = $O(n^2)$

16 Quick Sort Complexity Summary

Case	Recurrence	Time Complexity
Best	$2T(n/2) + n$	$O(n \log n)$
Average	$2T(n/2) + n$ (balanced model)	$O(n \log n)$
Worst	$T(n-1) + n$	$O(n^2)$